

Bộ bản in nộp kèm — 22 câu

File này chỉ dùng để chuẩn bị **bản in**, không phải đáp án học thuộc. In từ trước, ghi số câu thật lớn trên từng tập. Trong 10 phút viết A4 không được xem; sau đó có 2 phút chọn tập liên quan.

Trạng thái

- **Đã có giao diện/cây source thật:** câu 4, 6, 8, 9, 11, 13, 15, 17, 20, 21 và 22.
 - **Trang câu lệnh:** câu 1, 2, 3, 5, 7, 10, 12, 14 và 18.
 - **Đề không yêu cầu bản in riêng:** câu 16 và 19.
 - Các UI trong file được tạo từ hai demo chạy được: observability-demo và print-artifacts-demo.
-

Câu 1 — Kiểm tra chất lượng và triển khai Microservices

```
docker compose up --build -d
docker compose ps
curl http://localhost:8081/availability
# Ví dụ công cụ tạo tải; đề không bắt buộc in kết quả load test
wrk -t2 -c20 -d30s http://localhost:8081/availability
```

Nộp trang lệnh này và, nếu có, một ảnh giao diện/cửa sổ terminal của công cụ kiểm tra.

Câu 2 — Cài đặt mã nguồn Microservices

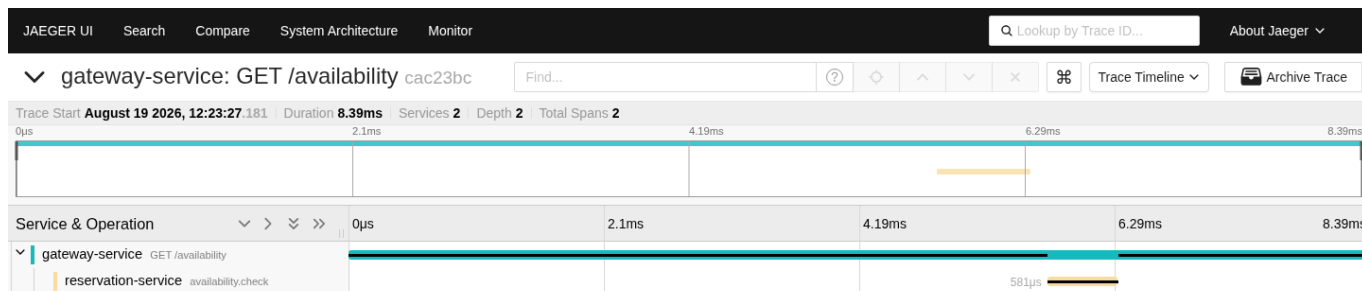
```
git clone https://github.com/khuongduy354/onthi-swe.git
cd onthi-swe/observability-demo
docker compose build gateway reservation
docker compose up -d
docker compose ps
```

Câu 3 — Thiết lập và thực hiện scaling

```
docker compose up -d --scale reservation=3
docker compose ps
# Hoặc Kubernetes:
kubectl scale deployment reservation --replicas=3
kubectl get pods
```

Câu 4 — Logging và tracing Microservices

```
cd observability-demo
docker compose up --build -d
curl http://localhost:8081/availability
docker compose logs --no-log-prefix gateway reservation
```



Trong ảnh: **2 services, 2 spans** — gateway-service: GET /availability gọi reservation-service: availability.check.

Câu 5 — Cài đặt component/service mới

```
cd observability-demo
docker compose build consumer
docker compose up -d consumer
docker compose ps consumer
docker compose logs consumer
```

Câu 6 — Micro-Frontends

Hai Micro-Frontend chạy riêng:

Account Micro-Frontend

This interface can run and be deployed independently.

Account MFE - port/module A

Account

Nguyễn An

an@example.test · Standard plan

Manage profile

Search Micro-Frontend

This interface can run and be deployed independently.

Search MFE - port/module B

Hotel search

Da Nang

12 hotels available

Search

App Shell tải và ghép ba UI fragment lúc runtime:

Composed Micro-Frontend system

App Shell composes three independently owned interface fragments at runtime.

App Shell

loads →

Independent UI fragments

Account MFE · port/module A

Account

Nguyễn An

an@example.test · Standard plan

Manage profile

Search MFE · port/module B

Hotel search

Da Nang

12 hotels available

Search

Report MFE · port/module C

Monthly bookings

1,284

↑ 12.4% from last month

Cấu trúc thể hiện Shell, các remote độc lập và phần dùng chung:

```
microfrontends/  
├── app-shell/  
│   ├── remote-manifest.json  
│   └── shell.js  
├── remotes/  
│   ├── account/index.html  
│   ├── search/index.html  
│   └── report/index.html  
└── shared/  
    └── design-tokens.css
```

Câu 7 — Triển khai Micro-Frontends

```
npm ci  
npm test  
npm run build  
npx vercel --prod
```

Câu 8 — JAMstack

Giao diện JAMstack được render từ file có sẵn trong dist/:

JAMstack static site

Markup was generated at build time; dynamic data would be fetched from an API.

Git/CMS → Build → Static HTML → CDN

Pre-rendered Markup

Architecture Notes

Fast architecture notes delivered as pre-rendered markup.

Served from static build output

Cây mã nguồn thật:

```
jamstack/  
├── src/  
│   ├── content/site.json  
│   └── pages/index.html  
├── scripts/build.py  
└── dist/index.html
```

Câu 9 — RAG

Giao diện hỏi–đáp, top-2 chunks và citation:

Local RAG interface

Keyword retrieval over local documents; answer includes the retrieved sources.

Question

How does Kappa recover and recompute reports?

Grounded answer

Kappa recovers or recomputes results by replaying its durable event log through the same stream-processing pipeline.

Retrieved top-2 citations

kappa.md#replay

Kappa reprocesses historical data by replaying the durable event log through the same stream processor.

event-sourcing.md#audit

Event Sourcing stores immutable domain events and rebuilds state by applying events in version order.

Cây mã nguồn thật:

```
rag/
├── data/
│   └── documents.py
├── ingestion/
│   ├── chunker.py
│   ├── embedder.py
│   └── indexer.py
├── infrastructure/
│   └── vector_store.py
├── retrieval/
│   ├── tokenizer.py
│   └── retriever.py
└── generation/
    ├── prompt_builder.py
    └── answer_service.py
```

Demo offline dùng embedding/retrieval xác định và answer mẫu có citation; `answer_service.py` là boundary để thay bằng LLM API khi có model/key.

Câu 10 — Triển khai RAG

Chọn **một** trong hai: trang lệnh dưới đây hoặc ảnh giao diện công cụ triển khai.

```
cd print-artifacts-demo
docker build -t rag-demo .
docker run -d --name rag-demo -p 8090:8090 rag-demo
curl http://localhost:8090/rag
```

Câu 11 — LLM-based Agent

Giao diện task, tool calls, tool results và final answer:

LLM-based Agent pattern

Offline MockLLMPlanner chooses tools; permissions, steps and results remain visible.

Task

Check two room-nights and calculate the total

Agent execution

Tool: check_inventory

args={'item': 'room-night'}

result={'item': 'room-night', 'available': True, 'units': 8}

Tool: calculate_total

args={'price': 120.0, 'quantity': 2}

result=240.0

Final answer: Two room-nights are available. Total: \$240.00.

Cây mã nguồn thật:

```
agent/
├── models/
│   ├── mock_llm.py          # planner offline
│   └── ollama_client.py     # adapter cho model thật
├── core/
│   ├── state.py             # task, max_steps, steps, answer
│   └── executor.py          # agent loop
├── tools/
│   ├── inventory.py
│   ├── pricing.py
│   └── registry.py          # tool name → function
├── memory/
│   └── checkpoint_store.py
└── policies/
    └── permissions.py       # allow/deny tool call
```

Lưu ý trung thực: demo offline dùng MockLLMPlanner; tools và agent loop chạy thật. Khi có API key/model local, thay planner bằng LLM mà không đổi tool contracts.

Câu 12 — Triển khai LLM-based Agent

Chọn **một** trong hai: trang lệnh dưới đây hoặc ảnh giao diện công cụ triển khai.

```
cd print-artifacts-demo
docker build -t agent-demo .
docker run -d --name agent-demo -p 8090:8090 agent-demo
curl http://localhost:8090/agent
```

Câu 13 — Event Sourcing

Giao diện nhập command StudentAdded; hệ thống append event và projector tạo Read Model:

Event Sourcing — input

A command appends an immutable event; a projector rebuilds the read model.

Student ID

Full name

Append StudentAdded event

Event Store

Append-only command flow

Command → Event Store → Projector

Existing events: 2

Cây mã nguồn thật:

```
event_sourcing/
├── domain/
│   ├── events.py
│   └── student.py      # apply event vào aggregate state
├── application/
│   ├── command_service.py
│   └── query_service.py
├── infrastructure/
│   └── event_store.py  # append/read immutable events
└── projections/
    └── student_projector.py
```

Câu 14 — Triển khai Event Sourcing

Chọn **một** trong hai: trang lệnh dưới đây hoặc ảnh giao diện công cụ triển khai.

```
cd print-artifacts-demo
docker build -t event-sourcing-demo .
docker run -d --name event-sourcing-demo -p 8090:8090 event-sourcing-demo
curl http://localhost:8090/event-sourcing/list
```

Câu 15 — Danh sách Event Sourcing

Danh sách được đọc qua `query_service` → `project_students()`, không replay trực tiếp trong giao diện:

Event Sourcing — student list

Query reads the projected Read Model, not the Event Store.

STUDENT ID	NAME	ARCHITECTURE SCORE
SV001	Nguyễn An	8.5

Câu 16 — Lưu trữ và replay Event Sourcing

Đề không ghi yêu cầu nộp kèm bản in riêng.

Câu 17 — Event-Driven

Giao diện publish ReservationCreated, kèm eventId, consumer và kết quả xử lý:

Event-Driven — publish input

Producer validates input, emits an event, and an independent consumer processes it.

Hotel ID

Publish ReservationCreated

Latest event
ReservationCreated
eventId: 36d2985c
Consumer: notification-service
Status: processed

Producer → event → Broker → event → Consumer

Cây mã nguồn thật:

```
event_driven/
├── contracts/
│   └── events.py          # ReservationCreated schema
├── producers/
│   └── reservation.py
├── infrastructure/
│   ├── in_memory_broker.py
│   └── dead_letter_queue.py
└── consumers/
    └── notification.py    # idempotent handler
```

Câu 18 — Triển khai Event-Driven

Chọn **một** trong hai: trang lệnh dưới đây hoặc ảnh giao diện công cụ triển khai.

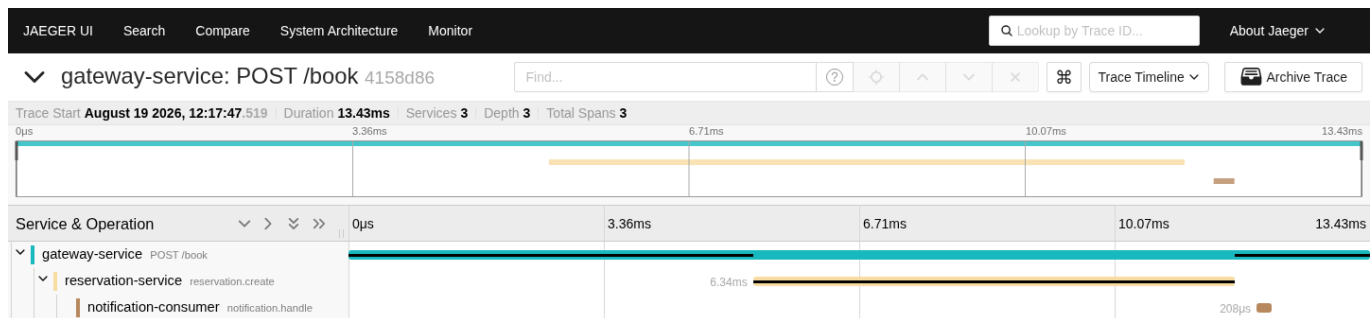
```
cd print-artifacts-demo
docker build -t event-driven-demo .
docker run -d --name event-driven-demo -p 8090:8090 event-driven-demo
curl http://localhost:8090/event-driven/input
```

Câu 19 — Nhập dữ liệu Event-Driven

Đề không ghi yêu cầu nộp kèm bản in riêng.

Câu 20 — Observability Event-Driven

```
cd observability-demo
docker compose up --build -d
curl -X POST http://localhost:8081/book \
  -H 'Content-Type: application/json' \
  -d '{"hotelId":"hotel-01","userId":"user-01"}'
docker compose logs --no-log-prefix gateway reservation consumer
docker compose ps
```



Trong ảnh: **3 services, 3 spans** — `POST /book` → `reservation.create` → `notification.handle`.

Redpanda console interface showing the `reservation-events` topic. The interface displays message details, including partition, offset, timestamp, key, and value. The value is a JSON object containing an `eventId`.

Cluster > Topics > **reservation-events**

642 B Size | 2 Messages | delete cleanup.policy | 7 days retention.ms | Infinite retention.bytes

Partition	Start Offset	Max Results	Filter	Actions
All	Newest - 50	50	☐	Search...

Offset	Partition	Timestamp	Key	Value
0	0	8/19/2026, 12:17:02 PM	⌚	{"eventId": "f0400ec4-0561-4e05-aeb4-5cb3dbac3cd9..."}
1	0	8/19/2026, 12:17:47 PM	⌚	{"eventId": "fa7968c6-acc0-4edb-b32e-a1ff7f4e1135..."}

10 / page |

Trong ảnh: topic `reservation-events`, partition/offset và JSON có `eventId`.

Câu 21 — Kappa

Giao diện thêm event vào durable stream:

Software Architecture Lab

Home RAG Events Kappa

Kappa — event input

New data enters the same durable stream used for replay.

Category

Hotel

Amount

150

Append event to stream

Kafka log

Single stream pipeline

Input → Log → Aggregate

Raw events retained: 5

Cây mã nguồn thật:

```
kappa/
├── producer/
│   └── input_service.py
├── storage/
│   ├── checkpoint_store.py
│   ├── event_log.py      # durable raw events + replay
│   └── serving_db.py     # materialized report rows
├── stream/
│   └── processor.py      # validate/deduplicate/aggregate
└── api/
    └── report_service.py
```

Câu 22 — Báo cáo Kappa

Giao diện báo cáo được tính từ stream events:

Kappa — near-real-time report

Stream processor deduplicates and aggregates; Report API reads the Serving DB.

Hotel

\$350

4 events

Flight

\$240

1 events

CATEGORY	COUNT	TOTAL
Hotel	4	\$350
Flight	1	\$240

Dữ liệu thô tạo ra đúng count và total trong báo cáo:

Kappa — raw report data

These retained events produce the aggregate report.

EVENT ID	TIME	CATEGORY	AMOUNT
e-001	09:00:02	Hotel	120
e-002	09:00:04	Hotel	80
e-003	09:00:07	Flight	240
e-004	09:00:09	Hotel	100
b27b65a4	now	Hotel	50